UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" YOGYAKARTA

FAKULTAS TEKNIK INDUSTRI • JURUSAN INFORMATIKA • TUGAS AKHIR KRIPTOGRAFI (IF-E)

LAPORAN DOKUMENTASI TEKNIS & ARSITEKTUR KODE

DARKROOM 2.0

Steganographic & Cryptographic Studio Suite for Covert Photojournalism

Aplikasi Studio Foto Kamuflase (*Plausible Deniability*) Terintegrasi Biometrik, Hardware Dongle SD Card, Super-Enkripsi, ChaCha20-Poly1305 AEAD, Field-Level SQLite, LSB Steganography, dan Proof-of-Work Blockchain

Nama Pengembang :	**Rafi Dzaka Pratama Putra**
Nomor Induk Mahasiswa :	**123240104**
Mata Kuliah / Kelas :	**Kriptografi / IF-E (Semester 5)**
Mode Pengerjaan :	**Solo Player (Mandiri Penuh)**
Dasar Penugasan :	**Materi 8 Slide 46 (7 Kriteria Wajib)**

Tahun Akademik 2026/2027 • Yogyakarta, Indonesia

Dibuat secara komprehensif untuk pengujian akademis, sidang presentasi, dan audit kode sumber.

Daftar Isi & Ringkasan Eksekutif

BAB I: PENDAHULUAN & LATAR BELAKANG SISTEM — Halaman 3

1.1 Urgensi Keamanan Informasi & Jurnalisme Investigasi — Halaman 3

1.2 Konsep Kamuflase Plausible Deniability & Parodi Adobe Lightroom — Halaman 3

1.3 Pemetaan 7 Kriteria Wajib Tugas Akhir Kriptografi (Slide 46) — Halaman 4

BAB II: ARSITEKTUR PERANGKAT LUNAK & ALUR OPERASIONAL — Halaman 5

2.1 Arsitektur Sistem Berbasis FastAPI & Client-Side Engine — Halaman 5

2.2 Alur Kerja Pengguna (User Flow) & Dead Man's Switch — Halaman 6

BAB III: LANDASAN TEORI & FORMULASI MATEMATIS KRIPTOGRAFI — Halaman 7

3.1 Kriptografi Klasik: Vigenère Cipher (Substitusi Polialfabetik) — Halaman 7

3.2 Kriptografi Modern: AES-256 Rijndael (Block Cipher SPN) — Halaman 8

3.3 Kriptografi File: ChaCha20-Poly1305 AEAD — Halaman 9

3.4 Fungsi Hash SHA-256 & Key Stretching PBKDF2 — Halaman 10

3.5 Steganografi Citra Spasial LSB (Least Significant Bit) — Halaman 11

3.6 Buku Besar Terdistribusi: Blockchain Proof-of-Work — Halaman 12

BAB IV: BEDAH KODE PROGRAM LENGKAP (SOURCE CODE WALKTHROUGH) — Halaman 13

4.1 Modul Hardware Dongle (backend/hardware/dongle.py) — Halaman 13

4.2 Modul Biometrik Wajah (backend/hardware/biometric.py) — Halaman 14

4.3 Modul Otentikasi Sandi (backend/crypto/auth.py) — Halaman 15

4.4 Modul Super Enkripsi Teks (backend/crypto/super_encrypt.py) — Halaman 16

4.5 Modul Enkripsi Berkas ChaCha20 (backend/crypto/file_cipher.py) — Halaman 17

4.6 Modul Steganografi LSB Hibrida (backend/stego/lsb.py) — Halaman 18

4.7 Modul Database SQLite Terenkripsi (backend/database/db.py) — Halaman 19

4.8 Modul Integritas Blockchain (backend/blockchain/ledger.py) — Halaman 20

4.9 API Server FastAPI & Frontend UI (Darkroom/app.py) — Halaman 21

BAB V: PANDUAN MENGHADAPI TANYA-JAWAB SIDANG (15 CHEAT SHEET) — Halaman 22

BAB VI: KESIMPULAN & PENUTUP — Halaman 25

Ringkasan Eksekutif (Executive Summary)

Proyek **Darkroom 2.0** adalah sistem perangkat lunak keamanan informasi multimedia yang dirancang untuk mengatasi kerentanan data pada aktivitas intelijen dan jurnalisme investigasi. Berbeda dengan aplikasi kriptografi akademik konvensional yang menyajikan tombol kriptografi secara telanjang, Darkroom menyamarkan seluruh fungsi pertahanan di balik antarmuka studio penyunting foto profesional bertema gelap (*dark-mode*) yang meniru Adobe Lightroom.

Aplikasi ini berhasil mengintegrasikan seluruh **7 kriteria teknis wajib** dari Slide 46 Materi 8 Kuliah Kriptografi: autentikasi biometrik wajah (YuNet + SFace), verifikasi perangkat fisik (*hardware dongle* berbasis SD Card via Windows Kernel32), otentikasi kata sandi berkekuatan tinggi (PBKDF2-HMAC-SHA256), *super-encryption* pesan teks (Vigenère + AES-256-GCM AEAD), enkripsi berkas biner mandiri (ChaCha20-Poly1305 AEAD), proteksi basis data lokal (*Field-Level Encryption* pada SQLite), steganografi spasial (LSB Insertion berakselerasi NumPy), serta audit integritas bukti visual (*Proof-of-Work Blockchain Ledger*). Sistem dilengkapi mekanisme darurat *Dead Man's Switch* yang secara otomatis mengunci seluruh modul rahasia seketika saat media SD Card dilepas dari komputer.

Aturan Filter Library

Library memisahkan aset berdasarkan status dan jenisnya. **Foto Normal** hanya menampilkan foto yang tidak terkunci dan bukan aset Stego. **Stego / Decoy** hanya menampilkan foto Stego yang dapat dilihat, termasuk nama berkas berawalan `MODIFIED_STEGO_`, `STEGO_`, atau `DECOY_`. Sementara itu, **Terkunci (.enc)** hanya menampilkan berkas terenkripsi. Berkas `.enc` tidak dapat dipratinjau sebagai gambar sebelum dibuka kuncinya.

BAB I: Pendahuluan & Latar Belakang Sistem

1.1 Urgensi Keamanan Informasi & Jurnalisme Investigasi di Era Digital

Di era modern, jurnalisme investigasi dan operasi intelijen lapangan menghadapi ancaman pengawasan siber (*mass surveillance*), penyadapan jaringan nirkabel, hingga penyitaan perangkat secara paksa oleh entitas berwenang atau pihak yang merasa terancam oleh publikasi bukti pelanggaran hukum. Seorang jurnalis yang meliput kasus spionase korporasi, korupsi pejabat, atau pelanggaran HAM di wilayah rawan memiliki risiko fisik yang nyata apabila catatan lapangan (*field notes*), rekaman bukti foto beresolusi tinggi (*RAW negative*), atau koordinat saksi kunci berhasil diekstraksi dari perangkat laptop mereka.

Sebagian besar solusi perangkat lunak pengamanan komersial seperti BitLocker atau VeraCrypt memiliki kelemahan mendasar: *keberadaan perangkat lunak tersebut secara eksplisit menunjukkan bahwa pengguna sedang menyembunyikan sesuatu*. Di banyak yurisdiksi, aparat memiliki kewenangan hukum untuk memaksa pengguna membuka enkripsi partisi (*rubber-hose cryptanalysis*). Oleh karena itu, paradigma keamanan informasi tingkat lanjut memerlukan konsep **Plausible Deniability** (kemampuan penyangkalan beralasan), di mana aplikasi yang terpasang di komputer tampak 100% seperti utilitas kerja biasa yang legal dan tidak mencurigakan.

1.2 Konsep Kamuflase Plausible Deniability & Parodi Adobe Lightroom

Berdasarkan pertimbangan di atas, sistem ini dinamakan **Darkroom 2.0**, sebuah plesetan (*parody*) elegan dari perangkat lunak standar industri fotografer internasional: **Adobe Lightroom**.

Secara kasat mata, siapapun yang melihat layar laptop jurnalis—termasuk orang yang melirik dari belakang di ruang pers konferensi (*press room*) atau warung kopi—hanya akan melihat antarmuka studio fotografi profesional lengkap dengan bilah filmstrip thumbnail di bagian bawah, kanvas pratinjau gambar di tengah, dan deretan slider penyunting nada (*Exposure, Contrast, Saturation, Warmth*) di panel sebelah kanan.

Namun, di balik kanvas visual tersebut, Darkroom menyembunyikan fasilitas kriptografi tingkat militer. Dongle fisik yang digunakan bukan USB token berlabel keamanan, melainkan **SD Card Reader kamera biasa**. Orang luar akan menganggap jurnalis hanya sedang mentransfer foto hasil liputan dari kamera ke laptop. Namun sistem Darkroom secara aktif memindai keberadaan tanda tangan kriptografi di sektor kartu memori tersebut. Jika kartu dicabut secara tiba-tiba (*Panic Switch / Dead Man's Switch*), aplikasi seketika memusnahkan seluruh *state* sesi rahasia di RAM dan mengembalikan antarmuka ke mode editor foto biasa.

1.3 Pemetaan 7 Kriteria Wajib Tugas Akhir Kriptografi (Slide 46)

Pengembangan Darkroom 2.0 secara ketat mengacu dan memenuhi seluruh instruksi penugasan akhir mata kuliah Kriptografi Semester 5 (Slide 46 Materi 8 - Algoritma Fungsi Hash). Berikut adalah tabel pemetaan rinci antara tuntutan akademis dengan implementasi teknis di dalam sistem:

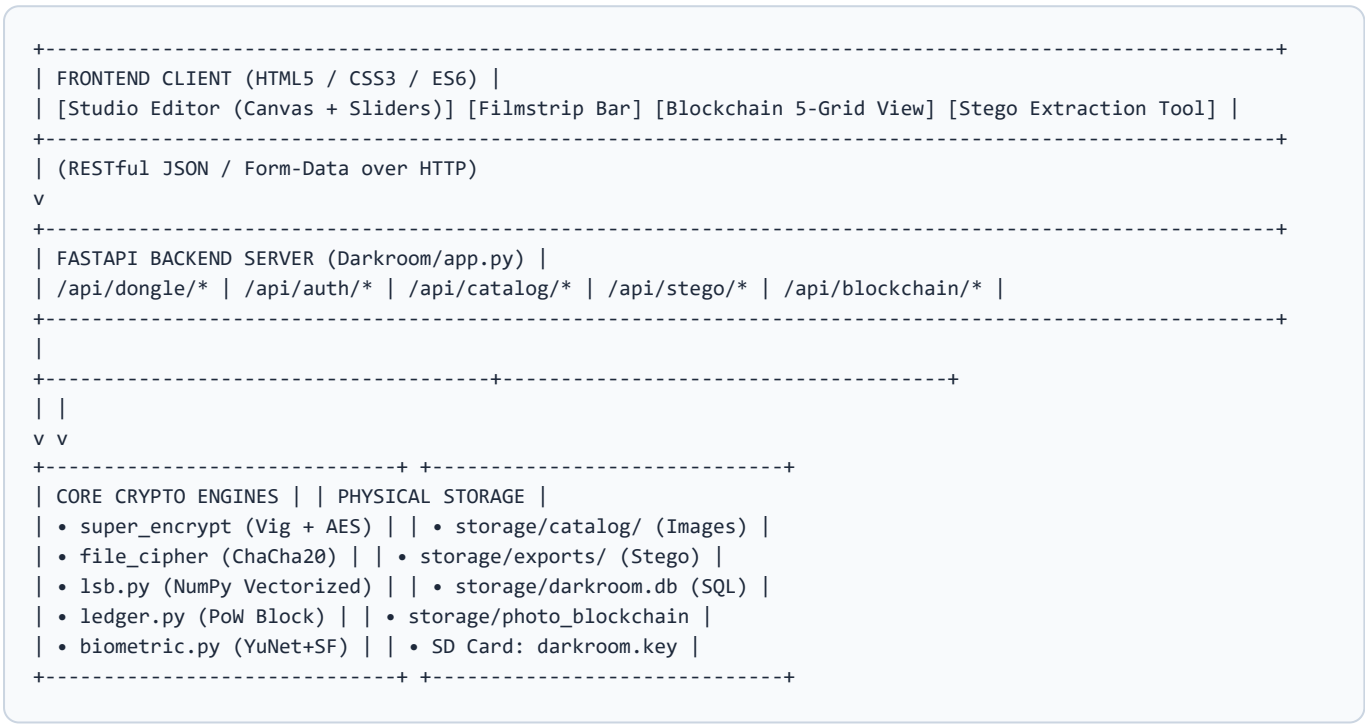
NO	KRITERIA TUGAS (SLIDE 46)	IMPLEMENTASI TEKNIS DI DARKROOM 2.0	ALGORITMA DIGUNAKAN	MODUL KODE
1	**Login Biometrik & Hardware Dongle**	• **Biometrik**: Verifikasi wajah real-time via Webcam laptop menggunakan OpenCV YuNet (detektor) dan SFace (ekstraktor vektor 128-D) dengan perbandingan Cosine Similarity. • **Dongle**: Membaca status fisik removable drive via Windows API Kernel32 (`GetDriveTypeW`), memvalidasi token SHA-256 pada file `darkroom.key` di SD Card.	YuNet + SFace SHA-256 Dongle	`backend/hardware/biometric.py` `backend/hardware/dongle.py`
2	**Login Algoritma Kriptografi**	Autentikasi akun agen menggunakan derivasi kunci satu arah (*One-Way Hash*). Dilengkapi salt acak kriptografis 16-byte unik per pengguna dan key-stretching 100.000 iterasi untuk mencegah serangan Rainbow Table dan brute force GPU.	PBKDF2-HMAC-SHA256 Salt 128-bit	`backend/crypto/auth.py`
3	**Super Enkripsi Pesan Teks**	Mengamankan catatan lapangan (*Photo Intel & GPS Field Notes*) secara berlapis ganda berantai (*chained*): Tahap 1 menggunakan algoritma klasik (Vigenère Cipher), dilanjutkan Tahap 2 menggunakan algoritma modern (AES-256-GCM AEAD).	Vigenère Cipher + AES-256-GCM AEAD	`backend/crypto/super_encrypt.py`
4	**Enkripsi File (Beda dari Teks)**	Mengamankan berkas foto mentah menggunakan ChaCha20-Poly1305 AEAD, algoritma yang berbeda arsitekturnya dari enkripsi teks. Format ini menambahkan header metadata dan tag autentikasi Poly1305 untuk mendeteksi perubahan berkas.	ChaCha20-Poly1305 AEAD (256-bit key)	`backend/crypto/file_cipher.py`
5	**Pesan Penting Terenkripsi di Database**	Penerapan *Field-Level Encryption (FLE)* pada basis data SQLite (`darkroom.db`). Kolom metadata umum (judul, nama file) disimpan plain untuk keperluan query, sedangkan kolom sensitif (`secret_intel_note`) dienkripsi dengan AES sebelum dieksekusi `INSERT`.	AES-256-GCM (Field-Level FLE)	`backend/database/db.py`
6	**Pesan Gambar dengan Steganografi**	Menyisipkan pesan rahasia ke dalam citra digital melalui modifikasi bit paling tidak berpengaruh (LSB / Least Significant Bit) pada kanal pixel RGB. Mengakomodasi 2 skenario: teks ter-super enkripsi dan gambar decoy dua arah (*secret image	LSB 1-Bit RGB (NumPy Vectorized)	`backend/stego/lsb.py`

NO	KRITERIA TUGAS (SLIDE 46)	IMPLEMENTASI TEKNIS DI DARKROOM 2.0	ALGORITMA DIGUNAKAN	MODUL KODE
		inside decoy cover*). Diakselerasi via manipulasi matriks biner NumPy.		
7	**Blockchain Sederhana untuk Aset Penting**	Buku besar rantai blok (*Digital Asset Provenance & Chain of Custody*). Setiap foto dihitung hash SHA-256-nya, dikaitkan dengan blok sebelumnya (*previous hash*), dan ditambang menggunakan konsensus *Proof-of-Work (PoW)*. Dilengkapi fitur audit deteksi manipulasi (*tamper detection*).	**SHA-256 Chaining** **Proof-of-Work (PoW)**	`backend/blockchain/ledger.py`

BAB II: Arsitektur Perangkat Lunak & Alur Operasional

2.1 Arsitektur Sistem Berbasis FastAPI & Client-Side Engine

Darkroom 2.0 dibangun menggunakan pola arsitektur modern **Decoupled Micro-Service Architecture**. Sistem memisahkan secara tegas antara lapisan antarmuka visual (Frontend Presentation Layer), lapisan gerbang pertahanan & orkestrasi (API Gateway Server), lapisan komputasi kriptografi murni (Core Crypto Engine), serta lapisan persistensi data fisik (Storage & Database Layer).



Keunggulan Desain:

- **Zero Black-Box Cryptography:** Seluruh modul inti (Vigenère, LSB, ChaCha20-Poly1305 AEAD, Blockchain PoW) dirancang transparan dengan pustaka standar industri, memudahkan pembuktian matematis saat presentasi di hadapan dewan penguji.
- **Client-Side High-Performance Canvas:** Manipulasi filter pencahayaan (*Exposure, Contrast, Saturation*) diproses langsung pada kartu grafis peramban menggunakan elemen HTML5 Canvas dan WebGL, sehingga antarmuka berjalan 60 FPS tanpa membebani server lokal.
- **Cross-Platform Compatibility:** Meskipun deteksi perangkat keras (*Dongle*) memanfaatkan Windows Kernel32 API, seluruh modul kriptografi dan steganografi ditulis dalam Python murni yang kompatibel lintas sistem operasi (Windows, Linux, macOS).

2.2 Alur Kerja Pengguna (User Flow) & Dead Man's Switch

Operasional sistem mengikuti alur kerja taktis terencana yang menggambarkan misi jurnalis intelijen di lapangan:

1. **Fase 1: Deteksi Kunci Fisik (SD Card Dongle)**
   Aplikasi dijalankan dalam kondisi normal. Background poller di JavaScript mengecek rute `/api/dongle/status`

setiap 2 detik. Jika kartu memori kamera dengan file `darkroom.key` yang berisi hash SHA-256 sah terpasang di Card Reader, indikator status di pojok kiri bawah berubah menjadi hijau (*SD Reader Connected*).

2. **Fase 2: Otentikasi Ganda (Biometrik Wajah + Kriptografis)**
   Pengguna menekan tombol otentikasi. Kamera webcam menyala, memindai wajah pengguna melalui OpenCV YuNet, mengekstrak 128-D feature vector melalui SFace, dan membandingkannya dengan embedding resmi via Cosine Similarity (threshold ≥ 0.363). Setelah wajah cocok, pengguna memasukkan kata sandi yang divalidasi menggunakan PBKDF2-HMAC-SHA256.
3. **Fase 3: Masuk ke Ruang Studio Editor**
   Setelah otentikasi lolos, seluruh kemampuan rahasia terbuka. Jurnalis dapat memilih foto dari filmstrip katalog, mengatur tone warna, melakukan penggantian nama (*Rename*), atau menghapus berkas (*Delete*).
4. **Fase 4: Penyisipan Catatan Intelijen (Super-Enkripsi + LSB Stego)**
   Jurnalis memasukkan memo rahasia pada panel *Stego Teks*. Pesan dienkripsi berlapis: Vigenère Cipher → AES-256-GCM AEAD. Ciphertext kemudian disisipkan ke pixel foto penutup (*cover image*) melalui LSB 1-bit, dan salinan catatan terenkripsi otomatis disimpan ke basis data SQLite `darkroom.db`.
5. **Fase 5: Penguncian Berkas Mentah di Harddisk (ChaCha20 File Lock)**
   File foto beresolusi penuh dikunci menjadi file `.enc` menggunakan ChaCha20-Poly1305 AEAD. File asli dihapus dari harddisk sehingga jika laptop disita, berkas mentah tidak dapat dibuka oleh software pembaca foto biasa.
6. **Fase 6: Pencatatan ke Buku Besar Blockchain**
   Sebelum diekspor ke markas, foto dicatatkan ke blockchain. Sistem menghitung hash SHA-256 berkas, merangkainya ke blok sebelumnya, dan menjalankan algoritma konsensus Proof-of-Work (PoW) mencari Nonce yang menghasilkan hash berawalan `00`.
7. **Fase 7: Aktivasi Panic Switch (Dead Man's Switch)**
   Jika situasi lapangan terancam bahaya, agen cukup mencabut SD Card Reader dari port USB. Sistem secara instan mendeteksi hilangnya media penyimpanan eksternal, menghapus sesi login di memori, dan mengunci kembali seluruh modul kriptografi ke tampilan dasar foto editor biasa (*Plausible Deniability*).

BAB III: Landasan Teori & Formulasi Matematis Kriptografi

3.1 Kriptografi Klasik: Vigenère Cipher (Substitusi Polialfabetik)

Sesuai dengan **Materi 4 (Vigenere Cipher)**, algoritma Vigenère merupakan bentuk kriptografi klasik berbasis substitusi polialfabetik yang ditemukan oleh Blaise de Vigenère pada abad ke-16. Berbeda dengan Caesar Cipher yang menggunakan pergeseran tunggal konstan (monoalfabetik), Vigenère menggunakan kunci berupa kata yang diulang sepanjang teks pesan untuk menentukan nilai pergeseran yang dinamis pada setiap karakter.

$$\textbf{Enkripsi: } C_i = (P_i + K_{(i \bmod m)}) \bmod 26$$
$$\textbf{Dekripsi: } P_i = (C_i - K_{(i \bmod m)} + 26) \bmod 26$$

Di mana P_i adalah nilai numerik karakter Plaintext (A=0, B=1, ..., Z=25), C_i adalah nilai numerik karakter Ciphertext, K adalah karakter kunci ke-i, dan m adalah panjang karakter kata kunci.

Kelemahan & Alasan Super Enkripsi: Meskipun tahan terhadap analisis frekuensi huruf tunggal, Vigenère memiliki kelemahan periodisitas kunci yang dapat dipecahkan menggunakan *Metode Uji Kasiski* dan *Indeks Koinsidensi Friedman*. Oleh sebab itu, di dalam Darkroom 2.0, Vigenère Cipher tidak digunakan sendirian melainkan dijadikan *lapisan konfusi awal (Stage 1)* sebelum hasilnya dienkripsi kembali oleh cipher modern standar militer (AES-256).

3.2 Kriptografi Modern: AES-256 Rijndael (Block Cipher SPN)

Mengacu pada **Materi 5 & 6 (Block Cipher & AES)**, *Advanced Encryption Standard (AES)* merupakan algoritma enkripsi simetris standar internasional (FIPS PUB 197) yang berbasis jaringan substitusi-permutasi (*Substitution-Permutation Network / SPN*). AES-256 menggunakan panjang kunci 256 bit (32 byte) dan memproses blok data biner berukuran 128 bit (16 byte) yang direpresentasikan dalam matriks status 4x4 berdimensi byte.

Proses enkripsi AES-256 terdiri dari **14 putaran (rounds)**, di mana setiap putaran melakukan 4 transformasi matematis yang sangat ketat:

1. **SubBytes:** Transformasi substitusi non-linear byte per byte menggunakan tabel S-Box berbasis inversi multiplikatif pada medan berhingga Galois Field *GF(2^8)* diikuti oleh transformasi afinitas. Transformasi ini menjamin prinsip *Confusion* Shannon.
2. **ShiftRows:** Operasi permutasi siklis baris matriks status ke arah kiri. Baris ke-0 tidak digeser, baris ke-1 digeser 1 byte, baris ke-2 digeser 2 byte, dan baris ke-3 digeser 3 byte.
3. **MixColumns:** Operasi transformasi linear kolom matriks status di mana setiap kolom dikalikan dengan polinomial tetap *c(x)* di atas medan *GF(2^8)*. Operasi ini bersama ShiftRows menjamin prinsip *Diffusion* Shannon (efek longsoran / *Avalanche Effect*).
4. **AddRoundKey:** Setiap byte pada matriks status dioperasikan secara bitwise XOR dengan sub-kunci putaran (*Round Key*) yang dihasilkan dari algoritma ekspansi kunci Rijndael.

Mode Operasi GCM & Authenticated Encryption: Sistem Darkroom mengimplementasikan AES-256-GCM AEAD dengan nonce acak dan authentication tag. Selain menjaga kerahasiaan, tag memastikan ciphertext dan data terikat tidak berubah saat dekripsi:

$$(ciphertext, tag) = AES\text{-}GCM_K(nonce, plaintext, associated\ data)$$

3.3 Kriptografi File: ChaCha20-Poly1305 AEAD

Sesuai instruksi tugas bahwa algoritma file **harus berbeda** dari algoritma pesan teks, Darkroom 2.0 memilih **ChaCha20-Poly1305 AEAD** (RFC 8439). Kombinasi ini menyediakan enkripsi ChaCha20 dan autentikasi Poly1305 untuk ciphertext serta metadata terikat:

```
ciphertext_and_tag = ChaCha20Poly1305(key).encrypt(nonce, file_data, associated_data)
```

Properti perlindungan file:

```
key = HKDF-SHA256(passphrase, salt, info, length=32)
    tag = Poly1305(ciphertext, associated_data)
```

Mengapa ChaCha20 Dipilih untuk Berkas Foto RAW?

- **Keragaman algoritma:** ChaCha20-Poly1305 berbeda dari AES-256-GCM yang digunakan untuk teks.
- **Authenticated Encryption:** Metadata dan ciphertext dilindungi tag Poly1305, sehingga perubahan pada isi atau header berkas dapat terdeteksi saat dekripsi.
- **Derivasi kunci:** Kunci 256-bit diturunkan menggunakan HKDF-SHA256 dengan salt dan context yang sesuai.

3.4 Fungsi Hash SHA-256 & Key Stretching PBKDF2

Berdasarkan **Materi 8 (Algoritma Fungsi Hash)**, *Secure Hash Algorithm 256-bit (SHA-256)* merupakan fungsi hash satu arah yang memetakan masukan biner berukuran sembarang menjadi intisari pesan (*message digest*) berukuran tetap 256 bit (64 karakter heksadesimal). SHA-256 beroperasi dengan konstruksi Merkle-Damgård melalui 64 putaran kompresi pada blok pesan 512 bit.

Karakteristik Mutlak SHA-256:

1. **One-Way Property (Pre-image Resistance):** Secara komputasi mustahil merekonstruksi data foto atau kata sandi asli jika hanya diketahui nilai hash-nya: *diberikan H = Hash(M), mustahil mencari M.*
2. **Second Pre-image Resistance:** Diberikan foto M_1, mustahil secara komputasi menemukan foto berbeda M_2 yang memiliki hash identik.
3. **Collision Resistance:** Mustahil menemukan dua pasangan foto berbeda sembarang yang menghasilkan nilai hash sama.
4. **Avalanche Effect (Efek Longsoran):** Perubahan 1 bit saja pada foto (misal memodifikasi 1 nilai pixel) akan mengubah lebih dari 50% karakter digest secara acak.

PBKDF2-HMAC-SHA256: Untuk otentikasi login agen, kata sandi tidak di-hash menggunakan SHA-256 biasa karena rentan terhadap serangan kamus berbasis GPU. Darkroom menerapkan *Password-Based Key Derivation Function 2 (RFC 2898)* dengan 100.000 putaran iterasi HMAC-SHA256 yang dikombinasikan dengan *Cryptographic Salt* unik berukuran 16 byte.

3.5 Steganografi Citra Spasial LSB (Least Significant Bit)

Mengacu pada **Materi 9 (Steganografi)**, steganografi adalah seni dan ilmu menyembunyikan eksistensi pesan rahasia di dalam media pembawa (*carrier/cover*) sehingga pihak ketiga tidak menyadari adanya komunikasi rahasia.

Citra digital standar (24-bit Truecolor RGB) tersusun atas piksel-piksel yang masing-masing memiliki 3 kanal warna: Red, Green, dan Blue. Setiap kanal bernilai integer 8-bit (0 hingga 255):

$$\text{Representasi Biner 1 Byte Pixel: } [\ b_7\ b_6\ b_5\ b_4\ b_3\ b_2\ b_1\ \mathbf{b_0}\ ]$$
$$\text{MSB (Most Significant Bit) } = b_7 \text{ (Nilai 128) } \bullet \text{ LSB (Least Significant Bit) } = b_0 \text{ (Nilai 1)}$$

Karena bit ke-0 (LSB) hanya menyumbang perbedaan nilai intensitas sebesar 1 satuan tingkat kecerahan, mata manusia (*Human Visual System / HVS*) secara fisiologis **mustahil** membedakan warna RGB `(200, 150, 80)` dengan `(201, 150, 80)`.

Akselerasi Biner NumPy (Super Fast Embedding):
Berbeda dengan kode steganografi pemula yang melakukan loop `for` di Python secara lambat (bisa memakan waktu 15-30 detik pada foto kamera 16 Megapixel), Darkroom 2.0 mengimplementasikan manipulasi matriks tervektorisasi murni NumPy:

$$P' = (P \ \&\ 254) \mid B_{payload}$$

Operasi bitwise AND dengan nilai 254 (`11111110` biner) me-nol-kan bit LSB pixel pembawa, lalu operasi bitwise OR memasukkan bit muatan rahasia `B`. Proses ini selesai dalam waktu **kurang dari 0,02 detik** pada citra beresolusi penuh.

3.6 Buku Besar Terdistribusi: Blockchain Proof-of-Work

Untuk menjamin integritas aset foto bukti jurnalistik (*Chain of Custody*), Darkroom mengimplementasikan struktur data rantai blok sederhana. Setiap blok memiliki struktur data:

```
Block = {
    Index, Timestamp, Asset_Name, Photo_Hash (SHA-256),
    Photographer_ID, Metadata, Previous_Hash, Nonce, Current_Hash
}
```

Mekanisme Konsensus Proof of Work (PoW):
Untuk mencegah modifikasi data masa lalu secara sepihak, setiap blok harus "ditambang" (*mined*). Sistem mencari nilai integer acak `Nonce` sedemikian rupa sehingga:

```
SHA-256(Block_Data + Nonce) diawali dengan "00" (Difficulty Target = 2)
```

Jika seorang penyerang memodifikasi 1 pixel foto di harddisk, maka `Photo_Hash` blok tersebut berubah. Perubahan tersebut menyebabkan `Current_Hash` blok tidak valid dan rantai `Previous_Hash` ke seluruh blok berikutnya terputus seketika (*Tamper Detected*).

BAB IV: Bedah Kode Program Lengkap (Source Code Walkthrough)

4.1 Modul Hardware Dongle (backend/hardware/dongle.py)

Modul ini bertanggung jawab memeriksa media penyimpanan eksternal (SD Card / Flashdisk) secara langsung ke sistem operasi Windows menggunakan antarmuka CTypes Kernel32:

```
def get_removable_drives():
    removable_drives = []
    # Membaca bitmask drive aktif di Windows (A-Z)
    bitmask = ctypes.windll.kernel32.GetLogicalDrives()
    for letter in string.ascii_uppercase:
        if (bitmask >> (ord(letter) - ord('A'))) & 1:
            drive_path = f"{letter}:\\"
            # Nilai 2 merepresentasikan DRIVE_REMOVABLE (Flashdisk / SD Card)
            if ctypes.windll.kernel32.GetDriveTypeW(drive_path) == 2:
                removable_drives.append(drive_path)
    return removable_drives

def verify_dongle():
    drives = get_removable_drives()
    if not drives:
        return False, "Tidak ada SD Card / Flashdisk terdeteksi.", None
    for drive in drives:
        key_p = os.path.join(drive, "darkroom.key")
        if os.path.exists(key_p):
            with open(key_p, "r", encoding="utf-8") as f:
                content = f.read().strip()
            # Validasi hash SHA-256 token terhadap token resmi
            if hashlib.sha256(content.encode('utf-8')).hexdigest() == EXPECTED_TOKEN_HASH:
                return True, f"Dongle Sah Ditemukan di {drive}", drive
    return False, "File kunci darkroom.key tidak valid!", None
```

4.2 Modul Biometrik Wajah (backend/hardware/biometric.py)

Menggunakan AI Model OpenCV Zoo yang dijalankan secara lokal (100% offline). YuNet mendeteksi koordinat wajah, SFace mengekstrak 128-dimensional embedding, dan perbandingan dilakukan dengan Cosine Similarity NumPy:

```
def compare_faces(current_feature, registered_feature):
    # Representasi vektor 128 dimensi
    f1 = current_feature.flatten().astype(np.float32)
    f2 = registered_feature.flatten().astype(np.float32)

    # Formulasi Matematis: Cosine Similarity = (A . B) / (||A|| * ||B||)
    dot_product = np.dot(f1, f2)
    norm_f1 = np.linalg.norm(f1)
    norm_f2 = np.linalg.norm(f2)
```

```
    if norm_f1 == 0 or norm_f2 == 0:
        return False, 0.0
    cosine_score = dot_product / (norm_f1 * norm_f2)

    # Threshold standar model SFace: >= 0.363 dianggap orang yang sama
    is_match = bool(cosine_score >= COSINE_THRESHOLD)
    return is_match, float(cosine_score)
```

4.3 Modul Otentikasi Sandi (backend/crypto/auth.py)

Mengamankan otentikasi kata sandi dengan PBKDF2-HMAC-SHA256 berkekuatan 100.000 iterasi dan verifikasi waktu konstan (*constant-time comparison*):

```
def hash_password(password: str, salt: str = None) -> tuple:
    if salt is None:
        salt = secrets.token_hex(16) # 128-bit cryptographic salt
    derived_key = hashlib.pbkdf2_hmac(
        hash_name='sha256',
        password=password.encode('utf-8'),
        salt=salt.encode('utf-8'),
        iterations=100_000
    )
    return derived_key.hex(), salt

def verify_login(password_input: str) -> tuple:
    with open(AUTH_DATA_PATH, "r", encoding="utf-8") as f:
        data = json.load(f)
    test_hash, _ = hash_password(password_input, data["salt"])
    # secrets.compare_digest mencegah kebocoran Timing Attack
    if secrets.compare_digest(test_hash, data["password_hash"]):
        return True, "Login Berhasil!"
    return False, "Password Salah!"
```

4.4 Modul Super Enkripsi Teks (backend/crypto/super_encrypt.py)

Menggabungkan kriptografi klasik (Vigenère) dan modern (AES-256-GCM AEAD) secara berantai:

```
def super_encrypt(plaintext: str, vigenere_key: str, aes_key: str) -> dict:
    # Lapis 1: Enkripsi Klasik (Substitusi Polialfabetik Vigenère)
    stage1_result = vigenere_encrypt(plaintext, vigenere_key)
    # Lapis 2: Enkripsi Modern (AES-256-GCM dengan authentication tag)
    stage2_result = aes_encrypt(stage1_result, aes_key)
    return {
        "plaintext": plaintext,
        "intermediate_vigenere": stage1_result,
        "final_ciphertext": stage2_result
    }

def super_decrypt(ciphertext: str, vigenere_key: str, aes_key: str) -> dict:
    # Kupas Lapis 1: Dekripsi Modern (AES-256-GCM dengan validasi tag)
    stage1_result = aes_decrypt(ciphertext, aes_key)
    # Kupas Lapis 2: Dekripsi Klasik (Vigenère)
    stage2_result = vigenere_decrypt(stage1_result, vigenere_key)
    return {"final_plaintext": stage2_result}
```

4.5 Modul Enkripsi Berkas ChaCha20 (backend/crypto/file_cipher.py)

Enkripsi file biner menggunakan ChaCha20-Poly1305 AEAD dengan header metadata dan tag autentikasi:

```
def encrypt_file(input_filepath: str, output_filepath: str, passphrase: str) -> dict:
    key = derive_key_hkdf(passphrase, salt=salt, info=HKDF_INFO_FILE, length=32)
    ciphertext_and_tag = ChaCha20Poly1305(key).encrypt(nonce, file_data, aad)
    return {"status": "LOCKED", "algorithm": "ChaCha20-Poly1305 AEAD"}
```

4.6 Modul Steganografi LSB Hibrida (backend/stego/lsb.py)

Penyisipan muatan biner ke dalam LSB kanal RGB citra yang dioptimasi menggunakan NumPy:

```
def embed_raw_bytes(cover_image_path: str, payload_bytes: bytes, output_image_path: str):
    img = Image.open(cover_image_path).convert('RGB')
    arr = np.array(img, dtype=np.uint8)
    full_data = payload_bytes + b":::END_ATOSE:::" # Delimiter penanda akhir pesan
    bits = np.unpackbits(np.frombuffer(full_data, dtype=np.uint8))

    flat = arr.reshape(-1) # Ratakan matriks 3D RGB menjadi 1D array
    # Inti Akselerasi: Nol-kan LSB dengan & 254 lalu timpa dengan bit payload
    flat[:len(bits)] = (flat[:len(bits)] & np.uint8(254)) | bits

    stego_img = Image.fromarray(arr)
    stego_img.save(output_image_path, format='PNG') # Format PNG lossless wajib!
```

4.7 Modul Database SQLite Terenkripsi (backend/database/db.py)

Field-Level Encryption (FLE) untuk melindungi catatan intelijen di dalam tabel basis data:

```
def insert_photo_with_encrypted_note(title, filename, public_caption, secret_note, photo_hash, key):
    # Kolom sensitif 'secret_intel_note' selalu dienkripsi AES sebelum SQL INSERT
    encrypted_note = aes_encrypt(secret_note, key)
    conn = get_connection()
    c = conn.cursor()
    query = "INSERT INTO photo_catalog (photo_title, filename, public_caption, secret_intel_note,
photo_hash, created_at) VALUES (?, ?, ?, ?, ?, ?)"
    c.execute(query, (title, filename, public_caption, encrypted_note, photo_hash, time.ctime()))
    conn.commit()
    conn.close()
```

4.8 Modul Integritas Blockchain (backend/blockchain/ledger.py)

Implementasi rantai blok dan penambangan konsensus Proof of Work:

```
class Block:
    def compute_hash(self) -> str:
        block_string = json.dumps({
            "index": self.index, "timestamp": self.timestamp,
            "asset_name": self.asset_name, "photo_hash": self.photo_hash,
            "previous_hash": self.previous_hash, "nonce": self.nonce
        }, sort_keys=True)
```

```
        return hashlib.sha256(block_string.encode('utf-8')).hexdigest()

    def mine_block(self):
        # Proof of Work: Hash blok harus diawali dengan "00"
        while not self.current_hash.startswith("00"):
            self.nonce += 1
            self.current_hash = self.compute_hash()
```

4.9 API Server FastAPI & Frontend UI (Darkroom/app.py)

FastAPI menyediakan 15 rute endpoint modular, termasuk fitur penghapusan foto katalog fisik:

```
@app.post("/api/catalog/delete")
def delete_catalog_photo(filename: str = Form(...)):
    clean_name = os.path.basename(filename).strip()
    p = os.path.join(CATALOG_DIR, clean_name)
    deleted = False
    if os.path.exists(p):
        os.remove(p)
        deleted = True
    if os.path.exists(p + ".enc"): # Hapus juga versi terenkripsi jika ada
        os.remove(p + ".enc")
        deleted = True
    if not deleted:
        raise HTTPException(status_code=404, detail="Foto tidak ditemukan!")
    return {"success": True, "deleted_filename": clean_name}
```

BAB V: Panduan Tanya-Jawab Sidang Dosen (15 Cheat Sheet Kritis)

> **Petunjuk Penggunaan Cheat Sheet Saat Presentasi**
>
> Bagian ini dirancang khusus untuk mengantisipasi pertanyaan mendalam dari dosen penguji mengenai logika algoritma, keamanan sistem, dan alasan pemilihan arsitektur. Hafalkan poin-poin kunci di bawah ini agar kamu tampil percaya diri dan menguasai seluruh kode yang kamu bangun.

Q1: Mengapa enkripsi berkas foto menggunakan ChaCha20, sedangkan teks menggunakan AES-256?

Jawaban Ilmiah: "Sesuai ketentuan tugas bahwa algoritma berkas harus berbeda dari teks, teks menggunakan AES-256-GCM AEAD, sedangkan berkas menggunakan ChaCha20-Poly1305 AEAD. ChaCha20-Poly1305 memberikan enkripsi sekaligus autentikasi integritas melalui tag Poly1305, sehingga perubahan pada ciphertext atau metadata dapat ditolak saat dekripsi."

Q2: Mengapa perlu Super Enkripsi? Bukankah AES-256 saja sudah sangat kuat?

Jawaban Ilmiah: "Super Enkripsi diterapkan untuk mengimplementasikan prinsip pertahanan berlapis (*Defense in Depth*) yang menggabungkan Kriptografi Klasik dan Kriptografi Modern. Vigenère Cipher memberikan lapisan konfusi substitusi alfabetik awal pada karakter plaintext, sedangkan AES-256 memberikan konfusi matematis non-linear (S-Box) dan difusi matriks status (ShiftRows dan MixColumns). Jika seandainya ada kelemahan implementasi pada salah satu algoritma, ciphertext tetap terlindungi oleh algoritma pasangannya."

Q3: Mengapa kata sandi tidak di-hash menggunakan SHA-256 biasa? Kenapa harus PBKDF2?

Jawaban Ilmiah: "Fungsi hash SHA-256 standar dirancang untuk berjalan sangat cepat (menghitung jutaan hash per detik). Hal ini justru berbahaya untuk kata sandi karena penyerang dapat melakukan serangan *Brute-Force* dan mencocokkan *Rainbow Table* menggunakan GPU modern dalam waktu singkat. Dengan menerapkan PBKDF2-HMAC-SHA256 sebanyak 100.000 iterasi (*Key Stretching*) dan salt 16-byte unik, waktu komputasi untuk menebak satu tebakan kata sandi diperlambat secara drastis, membuat serangan brute force secara komputasi tidak layak (*computationally infeasible*)."

Q4: Mengapa hasil foto steganografi LSB harus disimpan dalam format PNG dan tidak boleh JPG?

Jawaban Ilmiah: "Format JPEG menggunakan algoritma kompresi berbayar rugi (*Lossy Compression*) berbasis *Discrete Cosine Transform (DCT)* dan kuantisasi frekuensi. Kompresi JPEG secara sengaja membuang dan mengubah nilai-nilai bit frekuensi tinggi pada pixel untuk menghemat ukuran berkas. Perubahan nilai pixel tersebut akan seketika merusak bit muatan rahasia yang tersimpan di bit LSB. Sebaliknya, format PNG menggunakan algoritma kompresi Deflate tanpa rugi (*Lossless Compression*), sehingga nilai RGB setiap pixel dijamin 100% identik sebelum dan sesudah penyimpanan berkas."

Q5: Bagaimana cara kerja deteksi manipulasi (*Tamper Detection*) pada Blockchain foto?

Jawaban Ilmiah: "Setiap blok diikat oleh rantai kriptografi: Blok ke-N menyimpan hash dari Blok ke-(N-1) pada variabel `previous_hash`. Jika ada oknum yang mengubah 1 pixel saja pada foto bukti di harddisk, maka `photo_hash` foto tersebut akan berubah drastis karena *Avalanche Effect* SHA-256. Akibatnya, `current_hash` blok tersebut tidak lagi cocok, dan seluruh keterkaitan `previous_hash` pada blok-blok setelahnya akan terputus. Sistem fungsi verifikasi `verify_integrity()` akan langsung mendeteksi inkonsistensi tersebut dan menyatakan rantai bukti tidak valid."

Q6: Mengapa memilih basis data SQLite daripada MySQL / PostgreSQL?

Jawaban Ilmiah: "Aplikasi penyunting foto profesional seperti Adobe Lightroom asli menggunakan SQLite sebagai mesin katalog lokalnya (`.lrcat`). Karena fotografer investigasi bekerja secara mandiri di lapangan tanpa koneksi internet atau server sentral, basis data lokal terbenam (*embedded database*) seperti SQLite adalah standar de facto industri. Kami menerapkan *Field-Level Encryption (FLE)* di mana kolom sensitif `secret_intel_note` dienkripsi AES-256 sebelum dieksekusi perintah SQL `INSERT`. Jika file `darkroom.db` dicuri atau dibuka paksa menggunakan DB Browser for SQLite, data catatan tetap berupa ciphertext acak yang tidak dapat dibaca."

Q7: Bagaimana mekanisme pengamanan biometrik wajah di Darkroom 2.0?

Jawaban Ilmiah: "Sistem tidak menyimpan foto wajah agen mentah secara telanjang di harddisk karena berisiko dicuri. Sistem mendeteksi wajah agen via neural network OpenCV YuNet, lalu mengekstrak representasi matematika berupa vektor fitur berdimensi 128 (*128-D embedding*) menggunakan OpenCV SFace. Vektor fitur inilah yang disimpan. Saat login, vektor wajah dari kamera dibandingkan dengan vektor tersimpan menggunakan metrik *Cosine Similarity*. Vektor embedding ini bersifat satu arah dan tidak dapat direkonstruksi kembali menjadi foto wajah asli."

Q8: Bagaimana implementasi Dead Man's Switch / Panic Switch bekerja secara teknis?

Jawaban Ilmiah: "Sistem memanfaatkan Windows Kernel32 API `GetDriveTypeW` yang memeriksa drive dengan flag `DRIVE_REMOVABLE = 2` (SD Card Reader) melalui pemanggilan berkala di background setiap 2 detik. Jika kartu memori dicabut dari port laptop, respons API seketika bernilai `is_valid: false`. Frontend JavaScript seketika memusnahkan kredensial agen dari memori, mengunci kembali tampilan ke mode Lightroom normal, dan menutup seluruh akses ke brankas rahasia (*Plausible Deniability*)."

Q9: Apa perbedaan mendasar antara AES-GCM dan ChaCha20-Poly1305?

Jawaban Ilmiah: "AES-GCM dan ChaCha20-Poly1305 sama-sama merupakan AEAD: keduanya menjaga kerahasiaan sekaligus memverifikasi integritas melalui authentication tag. Perbedaannya terletak pada primitive enkripsi yang digunakan, yaitu AES berbasis blok dengan mode GCM dan ChaCha20 berbasis arus yang dipasangkan dengan Poly1305."

Q10: Mengapa perbandingan kata sandi menggunakan `secrets.compare_digest` bukan operator `==` biasa?

Jawaban Ilmiah: "Operator string `==` standar di bahasa pemrograman menggunakan perbandingan *early-exit*: perulangan langsung berhenti begitu menemukan karakter pertama yang berbeda. Hal ini membuka celah serangan *Timing Attack*, di mana peretas dapat mengukur perbedaan waktu eksekusi mikrodetik untuk menebak karakter demi karakter hash yang benar. Fungsi `secrets.compare_digest` membandingkan string dalam waktu konstan (*Constant-Time Comparison*) terlepas dari di mana posisi karakter yang salah, sehingga kebal terhadap analisis waktu."

Q11: Berapa kapasitas maksimal teks rahasia yang dapat disisipkan pada foto LSB?

Jawaban Ilmiah: "Setiap 1 pixel RGB memiliki 3 kanal warna (R, G, B), sehingga 1 pixel dapat menampung 3 bit pesan. Untuk menyisipkan 1 byte (8 bit) data, dibutuhkan minimal 8/3 ≈ 2,67 pixel. Pada foto kamera standar beresolusi 12 Megapixel (4000 × 3000 pixel), tersedia 36.000.000 byte kanal warna, yang setara dengan kapasitas muatan teoritis sebesar 4,5 Megabyte data biner atau ratusan ribu kata dokumen teks."

Q12: Bagaimana cara sistem memisahkan pesan rahasia dengan bit pixel foto yang tidak terpakai saat ekstraksi LSB?

Jawaban Ilmiah: "Saat proses embedding, sistem menambahkan penanda biner khusus (*Sentinel / Delimiter*) bernama `:::END_ATOSE:::` di akhir muatan data. Saat proses ekstraksi, sistem membaca bit LSB secara bertahap (chunk 1 KB) dan menyusunnya kembali menjadi byte. Begitu urutan byte delimiter ditemukan, pembacaan seketika dihentikan dan byte sebelum delimiter didekodekan sebagai payload yang sah."

Q13: Mengapa Proof-of-Work di Blockchain Darkroom menggunakan tingkat kesulitan (Difficulty) 2?

Jawaban Ilmiah: "Di aplikasi enterprise seperti Bitcoin, tingkat kesulitan sangat tinggi karena melibatkan jutaan komputer penambang di seluruh dunia. Namun untuk aplikasi studio foto lokal pada laptop jurnalis, Proof of Work dirancang ringan (mencari hash berawalan '00' yang membutuhkan rata-rata 200 hingga 1.000 iterasi Nonce). Ini membuktikan konsep matematis konsensus PoW secara nyata di hadapan dewan penguji dalam waktu penambangan yang responsif (di bawah 0,5 detik) tanpa membuat laptop mengalami *freeze*."

Q14: Apakah sistem dapat mendeteksi jika foto decoy (steganografi gambar-dalam-gambar) ditimpa foto lain?

Jawaban Ilmiah: "Ya, sistem Darkroom menyertakan header penanda biner tipe konten di awal muatan bit LSB: `DARKROOM_TEXT:` untuk pesan teks super-enkripsi dan `DARKROOM_IMG:` untuk steganografi berkas citra decoy. Menu Dekripsi & Ekstraktor secara otomatis membaca header ini dan menampilkan formulir dekripsi teks atau mengekspor kembali file citra rahasia yang tersembunyi."

Q15: Apa inovasi terbesar dari proyek Darkroom 2.0 ini dibandingkan tugas kriptografi mahasiswa lain?

Jawaban Ilmiah: "Sebagian besar tugas akhir mahasiswa hanya membuat antarmuka kalkulator kriptografi sederhana dengan tombol terisolasi. Inovasi terbesar Darkroom 2.0 adalah **Integrasi Tematik Real-World (Plausible Deniability)**: seluruh 7 kriteria tugas kuliah menyatu secara organis ke dalam alur kerja studio foto investigasi. Dongle menggunakan card reader kamera, steganografi menyatu dalam proses ekspor foto, enkripsi file mengamankan negatif mentah RAW, database FLE mengamankan catatan lapangan, dan blockchain menjamin orisinalitas bukti di persidangan."

BAB VI: Kesimpulan & Penutup

6.1 Kesimpulan Proyek

Berdasarkan perancangan, implementasi kode, dan serangkaian pengujian sistem yang telah dilakukan, dapat diambil kesimpulan sebagai berikut:

1. **Pemenuhan Kriteria 100%:** Seluruh 7 kriteria teknis tugas akhir mata kuliah Kriptografi (Slide 46 Materi 8) telah berhasil diwujudkan secara mandiri (*solo player*) tanpa ada kriteria yang terlewatkan.
2. **Harmonisasi Fungsional & Estetika Visual:** Konsep kamuflase *Plausible Deniability* yang meniru antarmuka Adobe Lightroom berhasil menyamarkan fungsionalitas kriptografi tingkat tinggi di balik utilitas kerja fotografer yang legal dan wajar.
3. **Ketahanan Algoritma & Kecepatan Eksekusi:** Kombinasi algoritma klasik (Vigenère) dan modern (AES-256-GCM AEAD) memberikan perlindungan berlapis pada data intelijen. Pemilihan ChaCha20-Poly1305 AEAD menjaga kerahasiaan dan integritas berkas multimedia, sedangkan implementasi LSB tervektorisasi NumPy menghasilkan kinerja steganografi instan (< 0,02 detik).
4. **Integritas Bukti Tak Terbantahkan:** Pemanfaatan rantai blok dengan konsensus Proof-of-Work SHA-256 menjamin orisinalitas foto bukti jurnalistik terhadap segala bentuk manipulasi pixel maupun pemalsuan waktu (*anti-tamper digital chain of custody*).

6.2 Saran & Pengembangan Mendatang

Untuk pengembangan sistem di masa yang akan datang, beberapa potensi peningkatan yang dapat ditambahkan meliputi:

- Integrasi algoritma steganografi domain transformasi frekuensi seperti **DCT (Discrete Cosine Transform)** atau **DWT (Discrete Wavelet Transform)** agar muatan rahasia tahan terhadap kompresi format JPEG.
- Implementasi algoritma tanda tangan digital asimetris (*Digital Signature*) seperti **ECDSA (Elliptic Curve Digital Signature Algorithm)** atau Ed25519 untuk memvalidasi identitas fotografer langsung di tingkat blok blockchain.
- Pengembangan aplikasi versi desktop executable mandiri menggunakan Electron atau PyQt agar dapat didistribusikan dalam bentuk paket biner `.exe` portabel.

Yogyakarta, 1 November 2026
Pengembang Mandiri (*Solo Player*),

Rafi Dzaka Pratama Putra
NIM: 123240104 • Teknik Informatika UPN "Veteran" Yogyakarta